BÁO CÁO DỰ ÁN

Đề tài:

Thiết kế Hệ thống tệp (File System) ảo mô phỏng kiến trúc FAT

Nhóm 2	
Thành viên	MSSV
Nguyễn Bá Hiếu	HE 210249
Nguyễn Đức Anh	HE 210342
Phạm Quang Minh	HE 210160
Nguyễn Bá Lâm	HE 200970
Vũ Tuấn Anh	HE200777

MỤC LỤC

PHẦN LÝ THUYẾT: Thiết kế Hệ thống tệp (File System) ảo mô phỏng kiến trúc FAT

1. Phương pháp cấp phát liên tục (Contiguous Allocation)

    1.1. Khái niệm

    1.2. Nguyên lý hoạt động

    1.3. Sơ đồ minh họa

    1.4. Ưu điểm và nhược điểm

    1.5. Ví dụ minh họa

2. Phương pháp cấp phát liên kết (Linked Allocation)

    2.1. Khái niệm

    2.2. Nguyên lý hoạt động

    2.3. Sơ đồ minh họa

    2.4. Ưu điểm và nhược điểm

    2.5. Cách truy cập và mở rộng tệp

3. Phương pháp cấp phát chỉ mục (Indexed Allocation)

    3.1. Khái niệm

    3.2. Nguyên lý hoạt động

    3.3. Sơ đồ minh họa

    3.4. Ưu điểm và nhược điểm

    3.5. Cách truy cập dữ liệu thông qua bảng chỉ mục

4. So sánh các phương pháp cấp phát và phân mảnh dữ liệu

    4.1. So sánh ba phương pháp cấp phát tệp

    4.2. Phân tích hai loại phân mảnh dữ liệu (Internal Fragmentation & External Fragmentation)

    4.3. Kết luận thực tế & Bài học kỹ thuật

5. Kiến trúc FAT (File Allocation Table)

    5.1. Khái niệm về FAT

    5.2 Vai trò của FAT

    5.3. Sơ đồ cấu trúc hệ thống tệp FAT

PHẦN LÝ THỰC HÀNH: Thiết kế Hệ thống tệp (File System) ảo mô phỏng kiến trúc FAT

1. Mục tiêu
2. Thiết kế ổ đĩa ảo
3. Các chức năng chính
4. Giao diện CLI
5. Code và kết quả mô phỏng
6. Kết luận

PHẦN LÝ THUYẾT: Thiết kế Hệ thống tệp (File System) ảo mô phỏng kiến trúc FAT

1. Khái niệm và Nguyên lý hoạt động

1.1. Khái niệm

Cấp phát liên tục đòi hỏi mỗi tập tin phải chiếm giữ một tập hợp các blocks liền kề nhau theo thứ tự địa chỉ tuyến tính trên đĩa từ hoặc ổ đĩa vật lý. Về mặt logic, tập tin là một chuỗi tuần tự 0, 1,... , n-1; khi được ánh xạ xuống đĩa, chuỗi logic này sẽ tương ứng trực tiếp với dải địa chỉ vật lý liên tiếp b, b+1,... , b+n-1.

1.2. Nguyên lý hoạt động

Bố trí không gian đĩa: Mỗi tệp tin chiếm dụng một dải các khối vật lý liên tiếp. Ví dụ: Một tệp data.txt có Start = 4 và Length = 3 sẽ chiếm trọn vẹn 3 khối liên tiếp là block 4,5,6 .

Quản lý thư mục: Để theo dõi tệp, bảng thư mục của hệ điều hành chỉ cần lưu 2 tham số số nguyên cho mỗi tệp tin:

Starting block: Địa chỉ của khối vật lý đầu tiên chứa dữ liệu.

Length: Tổng số khối vật lý liên tiếp mà tệp đó chiếm giữ.

Cơ chế truy xuất trực tiếp: Khi cần đọc khối logic thứ k của tệp tin, hệ điều hành không cần duyệt qua các khối trước đó mà tính trực tiếp địa chỉ khối vật lý theo công thức:

$$\text{Địa chỉ khối vật lý} = \text{Start} + k$$

1.3. Sơ đồ minh họa

Tên tệp (File Name)	Khối bắt đầu (Start)	Độ dài (Length)	Các khối chiếm dụng vật lý
count	0	2	0, 1
tr	14	3	14, 15, 16
mail	19	6	19, 20, 21, 22, 23, 24



1.4. Ưu điểm và nhược điểm

- Ưu điểm:
  + Tối ưu hóa hiệu năng I/O và giảm thiểu độ trễ cơ học
  + Hỗ trợ truy xuất trực tiếp với độ phức tạp $O(1)$
  + Tối ưu truy xuất tuần tự
  + Cấu trúc quản lý tại thư mục tối giản

- Nhược điểm:

  + Phân mảnh ngoài
  + Khó khăn trong việc mở rộng kích thước tệp
  + Lãng phí do phân mảnh trong

1.5. Ví dụ minh họa tính toán truy cập

Xét tệp tin email trong bảng thư mục có các thông số: Start = 19, Length = 6 (tương ứng với các khối logic $k \in \{0, 1, 2, 3, 4, 5\}$):

Truy xuất khối đầu tiên ($k = 0$):

$$Địa\ chỉ\ vật\ lý = 19 + 0 = 19$$

Hệ điều hành đọc trực tiếp tại khối đĩa số 19.

Truy xuất khối logic thứ 3 ($k = 3$):

$Địa\ Chỉ\ Vật\ Lý = 19 + 3 = 22$

Hệ điều hành không cần duyệt qua các khối 19, 20, 21 mà chuyển lệnh đọc trực tiếp đến khối đĩa số 22.

Truy xuất khối cuối cùng ($k = 5$):

$$Địa\ Chỉ\ Vật\ Lý = 19 + 5 = 24$$

Khối vật lý tương ứng là khối 24.

Kiểm tra giới hạn (Boundary Checking): Nếu một tiến trình yêu cầu truy cập khối logic $k \geq 6$, hệ điều hành đối chiếu với Length = 6 và ngay lập tức trả về lỗi vượt ngoài phạm vi tệp, đảm bảo tính toàn vẹn dữ liệu cho các tệp khác trên đĩa.

2. Khái niệm và nguyên lý hoạt động của Linked Allocation

2.1. Khái niệm

Linked Allocation là phương pháp cấp phát không gian đĩa, trong đó một tệp được lưu trữ dưới dạng chuỗi các khối dữ liệu (blocks) có thể nằm ở những vị trí không liên tiếp trên đĩa.

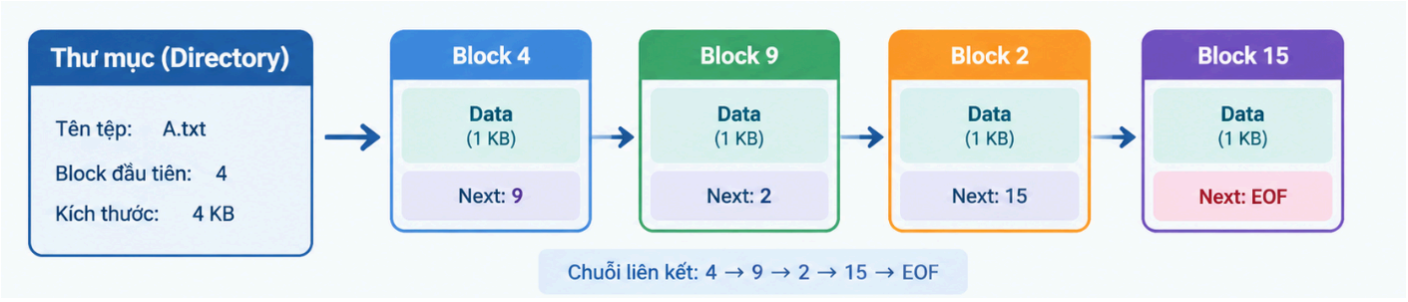
2.2. Nguyên lý hoạt động

Cấp phát: Hệ điều hành tìm các block trống để lưu dữ liệu của tệp.

Liên kết: Các block được nối với nhau bằng pointer, xác định thứ tự dữ liệu trong tệp.

Truy cập: Hệ điều hành bắt đầu từ block đầu tiên, lần theo các pointer để đọc dữ liệu cho đến khi gặp EOF.

2.3. Sơ đồ minh họa



2.4. Ưu điểm và nhược điểm của

- Ưu điểm:

+ Các block có thể nằm rải rác trên đĩa.
+ Khi mở rộng tệp chỉ cần cấp phát block mới và nối vào cuối chuỗi.

+ Hạn chế phân mảnh ngoài vì các block cùng một tệp không phải nằm cạnh nhau.

- Nhược điểm:

+ Truy cập ngẫu nhiên chậm do phải lần theo các pointer từ block đầu đến block cần đọc.
+ Tốn dung lượng lưu trữ vì mỗi block cần một phần dung lượng để lưu pointer.
+ Rủi ro khi pointer bị lỗi, có thể mất liên kết đến các block phía sau.
+ Các block nằm rải rác làm tăng chi phí truy cập, đặc biệt trên HDD.

2.5. Cách truy cập và mở rộng tệp

- Truy cập tệp (File Access)

+ Thư mục lưu địa chỉ block đầu tiên của tệp.
+ Hệ điều hành lần theo các pointer để tìm các block tiếp theo.
+ Quá trình tiếp tục cho đến khi gặp EOF.

- Mở rộng tệp (File Extension)

+ Cấp phát một block trống mới.
+ Cập nhật pointer của block cuối để trỏ đến block mới.
+ Đặt pointer của block mới thành EOF.
+ Cập nhật kích thước tệp và trạng thái cấp phát.

3. Phương pháp cấp phát chỉ mục (Indexed Allocation)

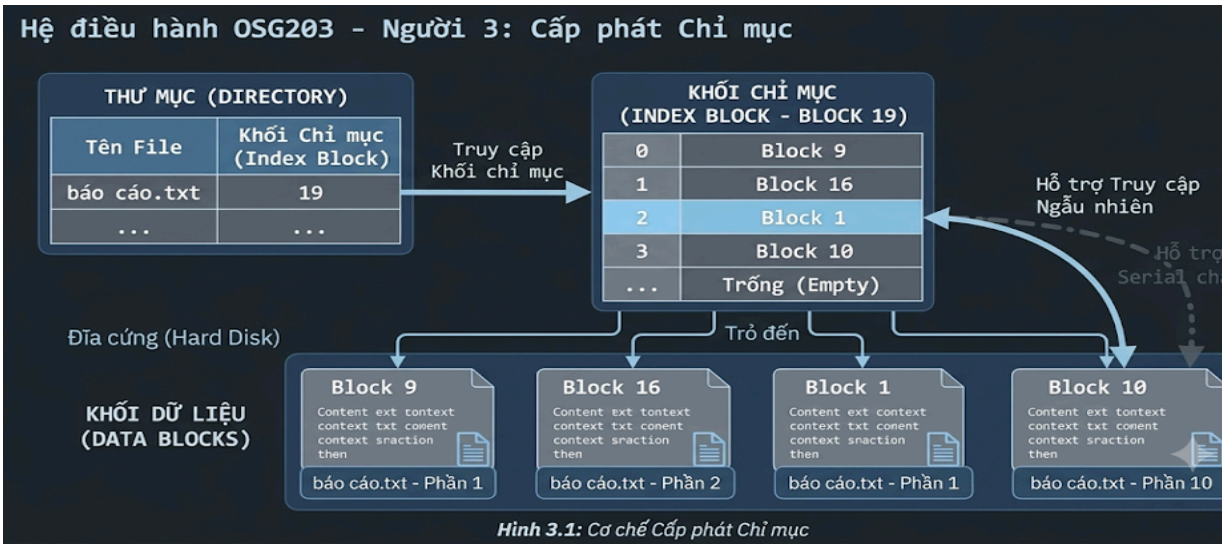
3.1 Khái niệm:

Indexed Allocation giải quyết vấn đề phân mảnh của cấp phát liên tục và việc truy cập ngẫu nhiên chậm của cấp phát liên kết bằng cách tập hợp tất cả các con trỏ (pointer) trỏ đến các khối dữ liệu (data block) của tập tin vào một khối duy nhất gọi là khối chỉ mục (index block)

3.2 Nguyên lý hoạt động:

- Mỗi tập tin sẽ có riêng một khối chỉ mục (Index Block).
- Khối chỉ mục này chứa một mảng/danh sách các địa chỉ đĩa, trong đó ô thứ *i* của khối chỉ mục trỏ trực tiếp đến khối dữ liệu thứ *i* của tập tin.
- Trong thư mục (Directory), thông tin lưu trữ của mỗi tập tin chỉ cần lưu tên tập tin và địa chỉ của khối chỉ mục.
- Khi muốn đọc khối thứ *k* của tập tin, hệ điều hành truy cập vào khối chỉ mục, lấy con trỏ ở vị trí thứ *k*, và trực tiếp đọc khối dữ liệu tương ứng.

3.3 Sơ đồ minh họa



Hình 3.1: Cơ chế Cấp phát Chỉ mục

3.4 Ưu điểm và nhược điểm:

- Ưu điểm:
  + Hỗ trợ truy cập trực tiếp/ngẫu nhiên nhanh.
  + Không bị phân mảnh ngoài, dễ mở rộng file.
  + Tự động mở rộng tập tin dễ dàng.
- Nhược điểm:
  + Tốn không gian đĩa cho khối chỉ mục (kể cả file nhỏ).
  + Giới hạn kích thước tập tin (nếu chỉ dùng 1 khối chỉ mục).

3.5 Cách truy cập dữ liệu bằng bảng chỉ mục:

- Tìm kiếm tập tin: hệ điều hành tra cứu trong Thư mục (Directory) để tìm tên tập tin cần truy cập và lấy ra địa chỉ của Khối chỉ mục (ví dụ: khối 19).
- Nạp khối chỉ mục: Nạp khối chỉ mục (Block 19) từ đĩa vào bộ nhớ RAM.
- Xác định vị trí khối dữ liệu:
  + Giả sử ứng dụng muốn đọc khối thứ k của tập tin (tính từ 0).
  + Hệ điều hành đọc giá trị tại chỉ số (index) thứ k bên trong khối chỉ mục để lấy địa chỉ đĩa tương ứng (ví dụ: muốn đọc khối 1→ tra index 1 thấy địa chỉ là 16).

- Đọc dữ liệu: Hệ điều hành di chuyển đầu đọc đến khối đĩa số 16 và đọc dữ liệu trả về cho ứng dụng.

- Linked scheme (Chỉ mục liên kết): Nối các khối chỉ mục lại với nhau bằng con trỏ.

- Multi-level index (Chỉ mục nhiều cấp): Khối chỉ mục cấp 1 trỏ đến các khối chỉ mục cấp 2, cấp 2 trỏ đến khối dữ liệu.

- Combined scheme (Unix inode): Sử dụng kết hợp các con trỏ trực tiếp (Direct Pointer), con trỏ gián tiếp đơn (Single Indirect), gián tiếp kép (Double Indirect) và gián tiếp ba (Triple Indirect).

4. So sánh các phương pháp cấp phát và phân mảnh dữ liệu

4.1 So sánh ba phương pháp cấp phát tệp

Tổng hợp cốt lõi các phương pháp lưu trữ đĩa và bài toán tối ưu bộ nhớ

Tiêu chí	Cấp phát Liên tục (Contiguous)	Cấp phát Liên kết (Linked)	Cấp phát Chỉ mục (Indexed / Inode)
Bản chất	Các khối dữ liệu nằm liên tiếp nhau trên đĩa.	Khối trước chứa con trỏ trỏ tới khối sau.	Sử dụng 1 khối chỉ mục gom tất cả địa chỉ khối dữ liệu.
Truy xuất	Rất nhanh (Cả tuần tự & ngẫu nhiên).	Chậm khi truy xuất ngẫu nhiên (phải duyệt từ đầu).	Nhanh (Hỗ trợ truy xuất trực tiếp tốt).
Mở rộng file	Khó (Cần vùng trống liên tục đủ lớn).	Dễ (Cấp khối trống bất kỳ rồi nối chuỗi).	Dễ (Cấp khối trống và chèn địa chỉ vào khối chỉ mục).
Phân mảnh	Bị Phân mảnh ngoài nghiêm trọng.	Chỉ bị Phân mảnh trong ở khối đĩa cuối cùng.	Chỉ bị Phân mảnh trong ở khối đĩa cuối cùng.

4.2 Phân tích hai loại phân mảnh dữ liệu (Internal Fragmentation & External Fragmentation)

- Phân Mảnh Trong (Internal Fragmentation)

+ Khái niệm: Lãng phí dung lượng *bên trong* khối đĩa đã được cấp phát cho tập tin.
+ Nguyên nhân: Kích thước khối đĩa là cố định (ví dụ: $4KB$), khối đĩa cuối cùng của file ít khi được lấp đầy hoàn toàn.
+ Ví dụ dễ nhớ:
    - File dung lượng $5000\ bytes$ → Cần cấp phát 2 khối đĩa $4KB$ (Tổng $8192\ bytes$).
    - Khối thứ nhất dùng hết $4096\ bytes$; khối thứ hai dùng $904\ bytes$.
    - → Thừa $3188bytes$ ở khối thứ hai bị bỏ trống và lãng phí.

- Phân Mảnh Ngoài (External Fragmentation)

+ Khái niệm: Tổng dung lượng bộ nhớ trống *đủ* để lưu tập tin mới, nhưng bị chia nhỏ thành nhiều vùng rời rạc nên không thể cấp phát cho tập tin yêu cầu vùng liên tiếp.
+ Nguyên nhân: Xảy ra do liên tục tạo, sửa, xóa tập tin khi áp dụng phương pháp *Cấp phát liên tục*.
+ Ví dụ dễ nhớ:
    - Tổng dung lượng đĩa trống là $10MB$, nhưng bị chia thành 5 khoảng trống nhỏ (mỗi khoảng $2MB$ không liền kề).
    - Một file mới dung lượng $4MB$ yêu cầu vùng trống liên tục → Hệ thống báo lỗi không đủ bộ nhớ.

4.3 Kết luận thực tế & Bài học kỹ thuật

- Chi phí dồn đĩa: Phân mảnh ngoài buộc hệ thống phải tốn chi phí thực hiện dồn đĩa (Defragmentation) để gộp vùng trống, gây tốn thời gian và giảm tuổi thọ đĩa/hiệu suất I/O.

- Ứng dụng hiện đại: Các Hệ điều hành hiện đại (Linux, Windows) chủ yếu sử dụng Cấp phát chỉ mục (Inode) hoặc Cấp phát liên kết cải tiến

(FAT32/NTFS) kết hợp khối đĩa cố định để loại bỏ hoàn toàn Phân mảnh ngoài và kiểm soát tốt Phân mảnh trong.

5. Kiến trúc FAT (File Allocation Table)

5.1. Khái niệm về FAT

FAT (File Allocation Table) là một hệ thống tệp dùng để quản lý cách lưu trữ và truy xuất dữ liệu trên các thiết bị như USB, thẻ nhớ và một số ổ đĩa.

FAT có nhiệm vụ theo dõi các cụm dữ liệu (cluster) mà mỗi tệp đang sử dụng, đồng thời xác định cụm tiếp theo của tệp nằm ở đâu.

Các phiên bản phổ biến gồm FAT12, FAT16 và FAT32.

5.2 Vai trò của FAT

- Quản lý vị trí lưu trữ của tệp

FAT giúp hệ điều hành xác định vị trí các cụm dữ liệu (Cluster) mà một tệp đang sử dụng. Nhờ đó, máy tính có thể tìm được dữ liệu cần thiết khi người dùng mở tệp.

Ví dụ: Một tệp văn bản được lưu tại các cụm 5, 9 và 12. FAT giúp hệ điều hành xác định những cụm này thuộc về cùng một tệp.

- Liên kết các cụm dữ liệu

Một tệp có thể được chia thành nhiều cụm và lưu ở những vị trí khác nhau trên thiết bị. FAT có nhiệm vụ ghi lại liên kết giữa các cụm, giúp hệ điều hành biết cụm hiện tại cần chuyển đến cụm nào tiếp theo.

Ví dụ: Cụm 5 liên kết với cụm 9, cụm 9 liên kết với cụm 12. Khi đọc tệp, máy tính sẽ lần lượt truy cập các cụm theo thứ tự này.

- Quản lý dung lượng lưu trữ

FAT theo dõi trạng thái của các cụm dữ liệu, bao gồm:

+ Cụm đã được sử dụng.
+ Cụm còn trống.
+ Cụm bị lỗi và không thể sử dụng.

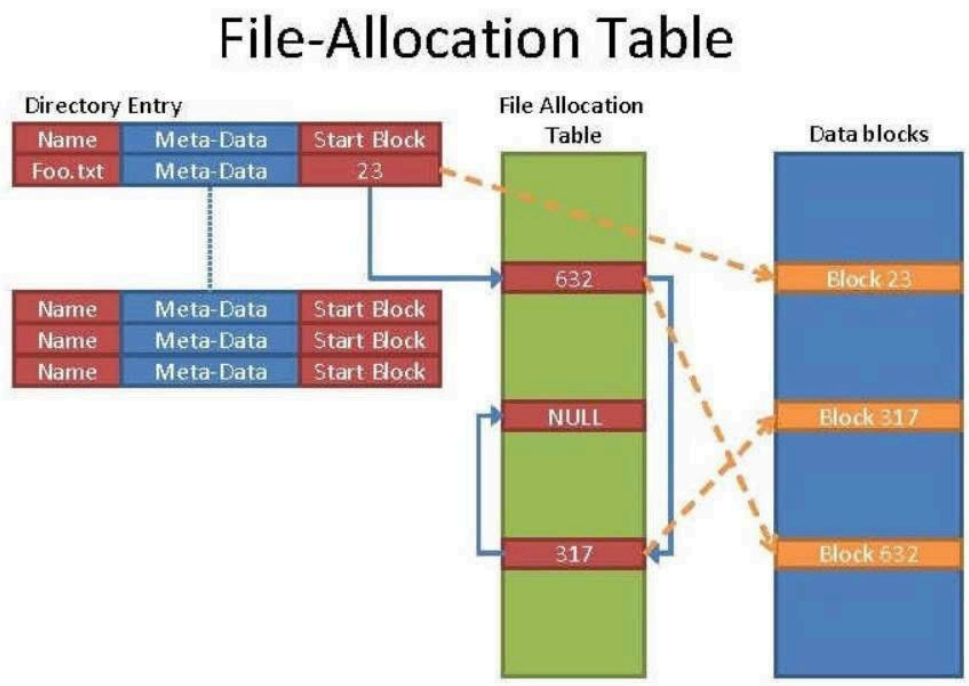
Khi người dùng lưu một tệp mới, hệ điều hành có thể tìm những cụm còn trống để phân bổ dung lượng lưu trữ.

- Hỗ trợ đọc và ghi dữ liệu

FAT giúp hệ điều hành tìm đúng vị trí dữ liệu khi mở tệp và cập nhật thông tin liên kết khi ghi hoặc chỉnh sửa tệp.

Ví dụ: Khi người dùng mở một tệp hình ảnh trong USB, hệ điều hành dựa vào thông tin thư mục và bảng FAT để tìm các cụm chứa hình ảnh, sau đó đọc dữ liệu để hiển thị lên màn hình.

5.3 sơ đồ FAT



Tìm hiểu FAT là gì?

PHẦN Thực hành: Thiết kế Hệ thống tệp (File System) ảo mô phỏng kiến trúc FAT

1. Mục tiêu

Xây dựng chương trình quản lý file trên ổ đĩa ảo disk.bin bằng Python, mô phỏng cơ chế hoạt động của hệ thống tệp FAT.

Chương trình hỗ trợ bốn chức năng chính: tạo file, ghi file, đọc file và xóa file.

2. Thiết kế ổ đĩa ảo

Ổ đĩa `disk.bin` có dung lượng 10MB, được chia thành ba vùng:

Vùng	Kích thước	Chức năng
Boot Region	512 byte	Lưu thông tin nhận diện
FAT Region	5.120 byte	Quản lý các cluster
Data Region	Phần còn lại	Lưu thông tin thư mục và nội dung file

Mỗi cluster có kích thước 4.096 byte, mỗi mục FAT chiếm 2 byte.

Các giá trị quy ước:

`FREE = 0`: Cluster trống.

`EOF = 65535`: Kết thúc chuỗi cluster.

`RESERVED = 65534`: Cluster dành riêng.

Cluster 0 được dành để lưu thông tin thư mục. Các cluster từ 1 trở đi dùng để lưu nội dung file.

3. Các chức năng chính

Khởi tạo ổ đĩa: Tạo file `disk.bin` và ghi thông tin nhận diện hệ thống tệp.

Quản lý FAT: Sử dụng `get_fat()` và `set_fat()` để đọc, cập nhật bảng FAT.

Cấp phát cluster: Hàm `allocate()` tìm cluster trống và đánh dấu cluster được sử dụng.

Giải phóng cluster: Hàm `release()` giải phóng chuỗi cluster của file.

Quản lý thư mục: Sử dụng `get_files()` và `save_files()` để lưu thông tin file dưới dạng JSON.

Tạo file: Hàm `create()` kiểm tra tên file và thêm file mới vào thư mục.

Ghi file: Hàm `write()` chia dữ liệu thành các cluster, ghi vào Data Region và liên kết chúng trong FAT.

Đọc file: Hàm `read()` lần theo chuỗi cluster để lấy nội dung file.

Xóa file: Hàm `delete()` xóa thông tin file và giải phóng các cluster đã sử dụng.

Khi ghi lại file, chương trình cấp phát và ghi dữ liệu mới trước khi giải phóng chuỗi cluster cũ. Nếu không đủ dung lượng, các cluster mới đã cấp phát sẽ được giải phóng.

4. Giao diện CLI

Chương trình sử dụng giao diện dòng lệnh để thực hiện các thao tác:

Lệnh	Chức năng

`create [ten_file]`	Tạo file
`write [ten_file] "noi_dung"`	Ghi nội dung
`read [ten_file]`	Đọc nội dung
`delete [ten_file]`	Xóa file

5. Code và kết quả mô phỏng

Phần code:

Thonny - D:\IA\Kỳ 2\OSG203\prj_osg203_final.py @ 1 : 1

File Edit View Run Tools Help

prj_osg203.py × prj_osg203_final.py * ×

```python
import os
import json
import shlex

# 1. Cau hinh o dia
DISK_SIZE = 10 * 1024 * 1024
CLUSTER_SIZE = 4096
BOOT = 512
FAT_SIZE = 5120
DATA = BOOT + FAT_SIZE
TOTAL = (DISK_SIZE - DATA) // CLUSTER_SIZE

FREE = 0
EOF = 65535
RESERVED = 65534

# 2. Khoi tao o dia ao
if not os.path.exists("disk.bin") or os.path.getsize("disk.bin") != DISK_SIZE:
    with open("disk.bin", "wb") as f:
        f.truncate(DISK_SIZE)

with open("disk.bin", "r+b") as f:
    f.seek(0)
    f.write(b"OSG203 FAT FILE SYSTEM")
    f.seek(BOOT)
    f.write(RESERVED.to_bytes(2, "little"))

# 3. Doc va ghi bang FAT
def get_fat(block):
    with open("disk.bin", "rb") as f:
        f.seek(BOOT + block * 2)
        return int.from_bytes(f.read(2), "little")

def set_fat(block, value):
    with open("disk.bin", "r+b") as f:
        f.seek(BOOT + block * 2)
        f.write(value.to_bytes(2, "little"))

# 4. Quan ly thu muc
def get_files():
    with open("disk.bin", "rb") as f:
        f.seek(DATA)
        data = f.read(CLUSTER_SIZE).split(b"\0")[0]
    return json.loads(data.decode()) if data else {}

```

```
45
46  def save_files(files):
47      data = json.dumps(files).encode()
48      if len(data) > CLUSTER_SIZE:
49          raise Exception("Thu muc da day!")
50      with open("disk.bin", "r+b") as f:
51          f.seek(DATA)
52          f.write(data.ljust(CLUSTER_SIZE, b"\0"))
53
54  # 5. Cap phat cluster
55  def allocate():
56      for block in range(1, TOTAL):
57          if get_fat(block) == FREE:
58              set_fat(block, EOF)
59              return block
60      return None
61
62  # 6. Giai phong chuoi cluster
63  def release(block):
64      while block != EOF and block != FREE:
65          next_block = get_fat(block)
66          set_fat(block, FREE)
67          block = next_block
68
69  # 7. Tao file
70  def create(name):
71      files = get_files()
72
73      if name in files:
74          print("File da ton tai!")
75          return
76
77      if len(name.encode()) > 32:
78          print("Ten file qua dai!")
79          return
80
81      files[name] = [-1, 0]
82      try:
83          save_files(files)
84          print("Tao file thanh cong!")
85      except Exception as e:
86          print(e)
87
```

```python
# 8. Ghi file
def write(name, content):
    files = get_files()

    if name not in files:
        print("Khong tim thay file!")
        return

    data = content.encode()
    blocks = []

    # Cap phat cluster moi
    for i in range(0, len(data), CLUSTER_SIZE):
        block = allocate()
        if block is None:
            for b in blocks:
                set_fat(b, FREE)
            print("Ghi file that bai!")
            return
        blocks.append(block)

    # Ghi du lieu
    try:
        with open("disk.bin", "r+b") as f:
            for i, block in enumerate(blocks):
                part = data[i * CLUSTER_SIZE:(i + 1) * CLUSTER_SIZE]
                f.seek(DATA + block * CLUSTER_SIZE)
                f.write(part.ljust(CLUSTER_SIZE, b"\0"))

        # Lien ket cac cluster
        for i in range(len(blocks) - 1):
            set_fat(blocks[i], blocks[i + 1])

        old_block = files[name][0]
        files[name] = [blocks[0] if blocks else -1, len(data)]
        save_files(files)

        # Xoa du lieu cu sau khi ghi thanh cong
        if old_block != -1:
            release(old_block)

        print("Ghi file thanh cong!")
```

```
    except Exception as e:
        if blocks:
            release(blocks[0])
        print("Ghi file that bai:", e)

# 9. Doc file
def read(name):
    files = get_files()

    if name not in files:
        print("Khong tim thay file!")
        return

    block, size = files[name]
    data = b""

    while block != -1 and block != EOF:
        with open("disk.bin", "rb") as f:
            f.seek(DATA + block * CLUSTER_SIZE)
            data += f.read(CLUSTER_SIZE)
        block = get_fat(block)

    print(data[:size].decode())

# 10. Xoa file
def delete(name):
    files = get_files()

    if name not in files:
        print("Khong tim thay file!")
        return

    block = files[name][0]
    if block != -1:
        release(block)

    del files[name]
    save_files(files)
    print("Xoa file thanh cong!")
```

```

# 11. Giao dien CLI
print("FAT FILE SYSTEM - 10 MB")
print("Lenh: create [ten_file], write [ten_file] \"noi_dung\", read [ten_file], delete [ten_file]")

while True:
   try:
       command = shlex.split(input("\nFAT> "))

       if not command:
           continue

       cmd = command[0]

       if cmd == "exit":
           break
       elif cmd == "create" and len(command) == 2:
           create(command[1])
       elif cmd == "write" and len(command) >= 3:
           write(command[1], " ".join(command[2:]))
       elif cmd == "read" and len(command) == 2:
           read(command[1])
       elif cmd == "delete" and len(command) == 2:
           delete(command[1])
       else:
           print("Lenh khong hop le!")

   except Exception as e:
       print("Loi:", e)

```

Kết quả mô phỏng:

```
Python 3.7.9 (bundled)
>>> %Run prj_osg203_final.py
 FAT FILE SYSTEM - 10 MB
 Lenh: create, write, read, delete, exit

 FAT> create hello.txt
 Tao file thanh cong!

 FAT> write hello.txt "hello world"
 Ghi file thanh cong!

 FAT> read hello.txt
 hello world

 FAT> delete hello.txt
 Xoa file thanh cong!

 FAT> read hello.txt
 Khong tim thay file!

 FAT>
```

6. Kết luận

Chương trình đã mô phỏng được cấu trúc ổ đĩa gồm Boot, FAT và Data; thực hiện các thao tác quản lý file cơ bản.